REST APIS

PROGRAMMING APPLICATIONS AND FRAMEWORKS (IT3030)

LEARNING OUTCOMES

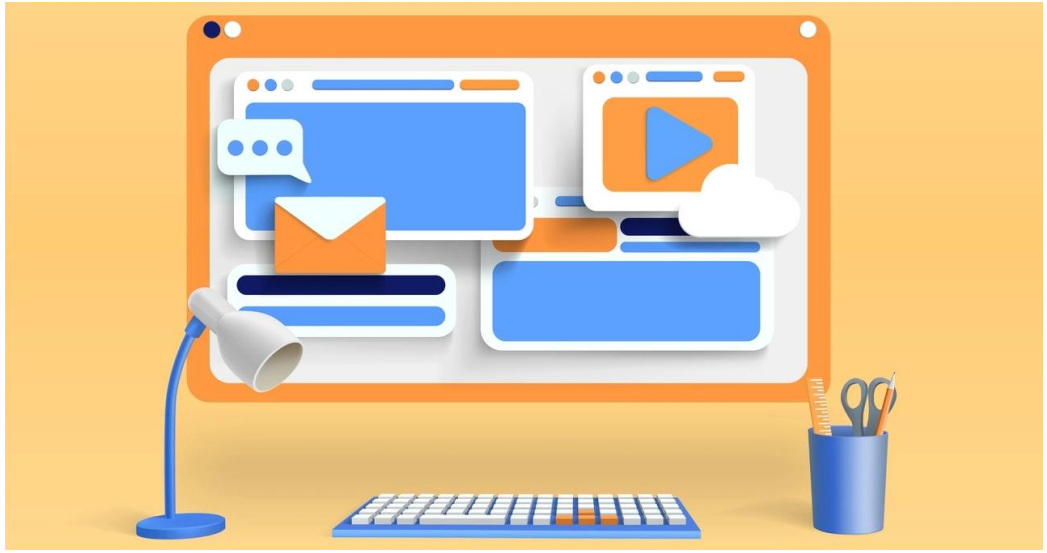
- After completing this lecture, you will be able to,
  - Describe the six REST architectural constraints.
  - Apply the knowledge gained in this lecture in designing proper REST APIs.
  - Evaluate if an API is a REST API or not.
  - Evaluate a given API to identify how much it conforms to the REST constraints using the Richardson Maturity Model.

CONTENTS

- Consuming web resources - Human vs. programs
- Application Programming Interface (API)
  - Examples
- REST
  - Six Architectural Constraints of REST
  - True REST vs. RPC (Remote Procedure Calls)
  - Richardson Maturity Model
- Some Facilitators of REST
  - HTTP, JSON, HAL
- Best Practices

CONSUMING WEB RESOURCES - HUMAN VS. PROGRAMS

- How do you consume web resources?
  - Web browsers
  - Web applications
  - Mobile applications etc.



Source: https://illuminatingfacts.com/a-timeline-of-the-rise-and-fall-of-popular-web-browsers/

CONSUMING WEB RESOURCES - HUMAN VS. PROGRAMS

- How do programs consume web resources?
  - Web APIs!
  - API stands for **Application Programming Interface**

Source: https://stileex.xyz/en/difference-computer-program-software/

APPLICATION PROGRAMMING INTERFACE (API)

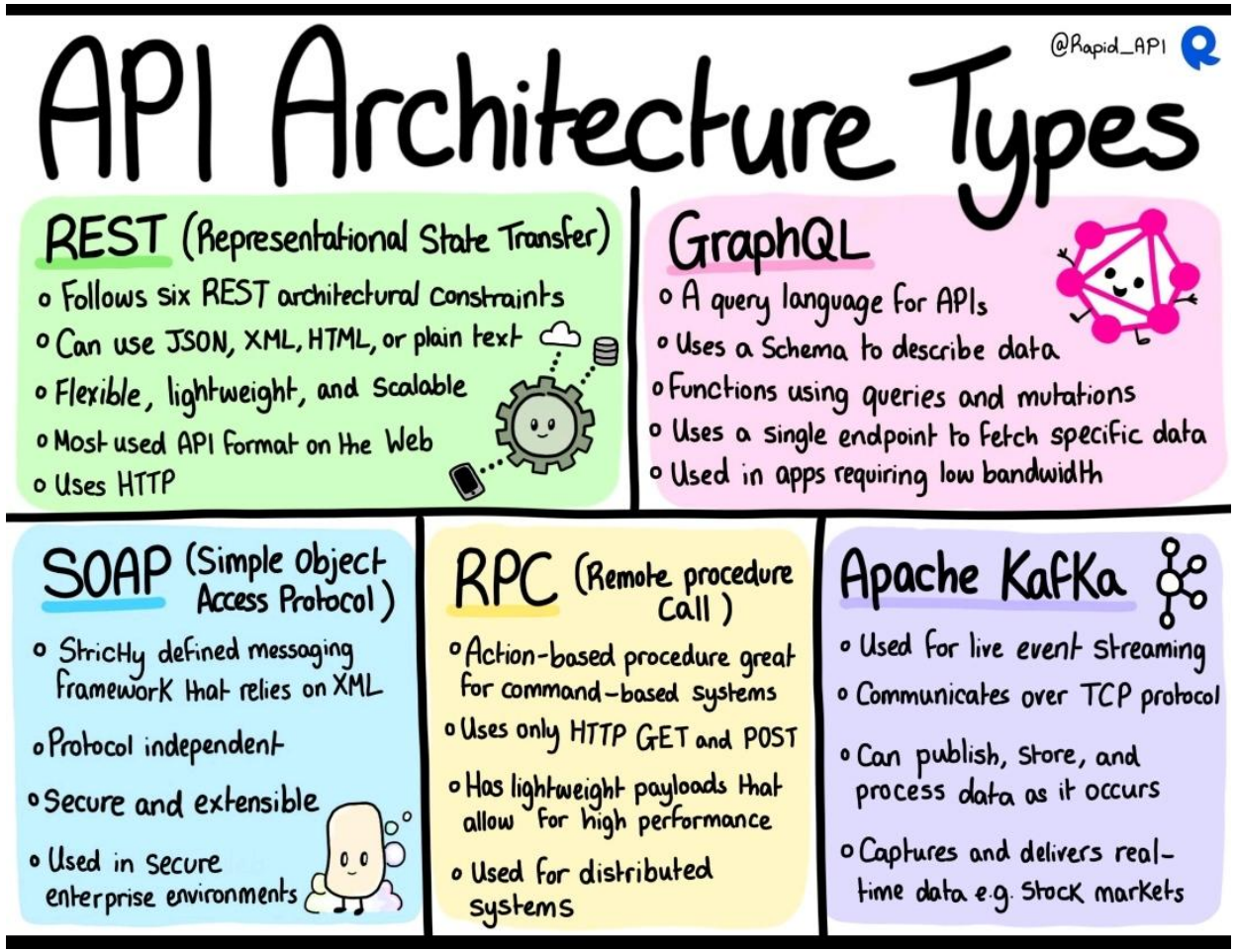
- What is an API?
  - A software interface which facilitates communication between two or more applications.
  - Abstracts away the complexities of application integrations.
  - A contract of services offered.

APPLICATION PROGRAMMING INTERFACE (API)

- Examples:
  - CORBA Implementations (Common Object Request Broker Architecture)
  - SOAP (Simple Object Access Protocol)
  - RPC (Remote Procedure Calls)
  - **REST** (Representational State Transfer)

Mostly legacy technologies

APPLICATION PROGRAMMING INTERFACE (API)



Source: https://ee.linkedin.com/posts/theo-borek-higgins_devops-api-devopslearn-activity-7028421145682944000-0YgL

REST (REPRESENTATIONAL STATE TRANSFER)

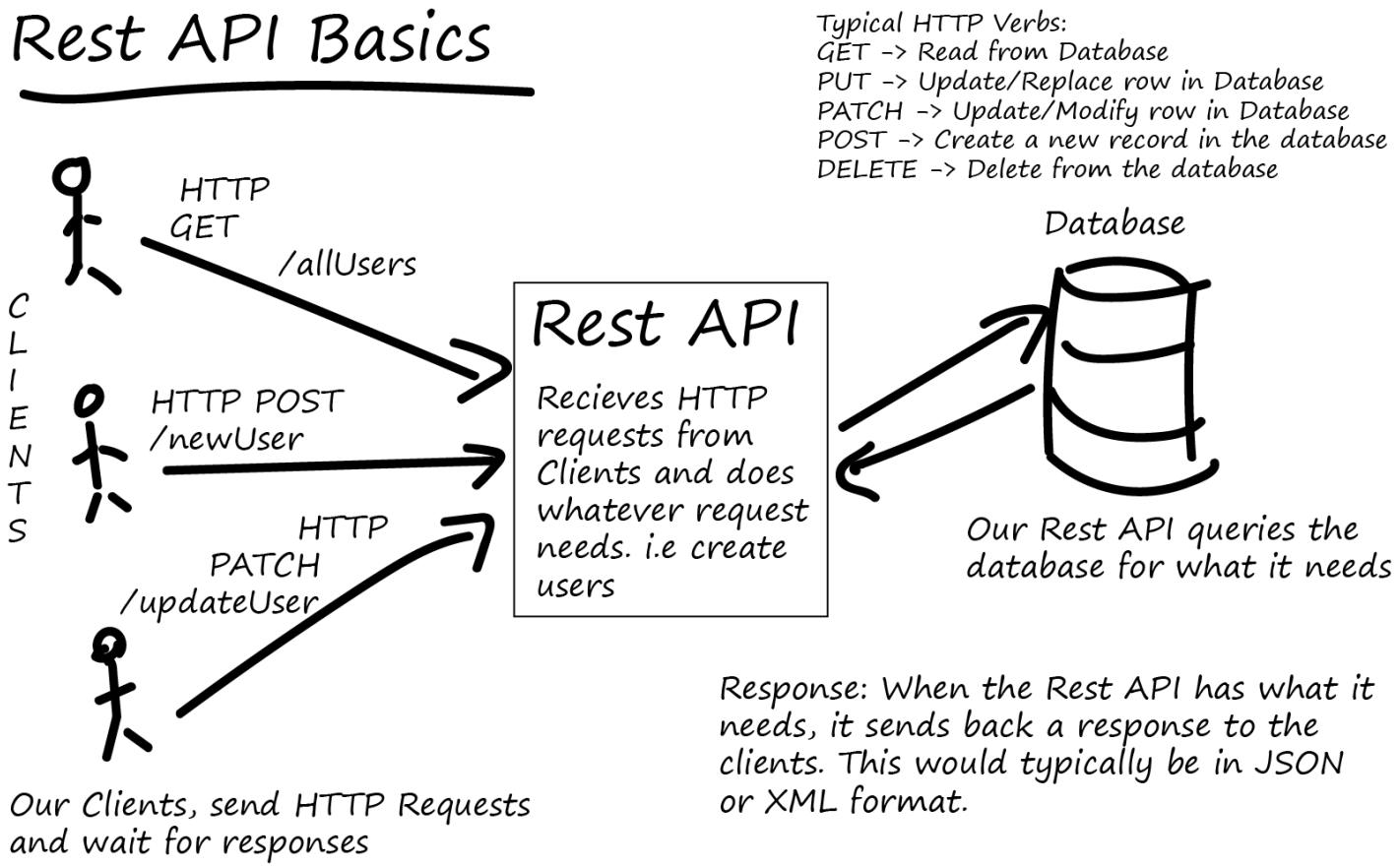
- What is REST?
  - an **architectural style** for creating web services.
  - Introduced by Roy Fielding in 2000 in his now famous Doctoral dissertation.
  - Arguably the most popular approach to creating Web APIs now.

Source: https://medium.com/@liams_o/15-fundamental-tips-on-rest-api-design-9a05bcd42920

REST (REPRESENTATIONAL STATE TRANSFER)

- Why is REST Popular?
  - Just an **architectural style** (not a technology or a full-blown protocol like SOAP)
  - If the API aligns with the six REST constraints, it is a REST API
  - No strict rules, very flexible
  - Simple to use and learn compared with previously used approaches (e.g., SOAP/ CORBA etc.)
  - Runs on HTTP Protocol

REST (REPRESENTATIONAL STATE TRANSFER)



Source: https://tutorialedge.net/software-eng/what-is-a-rest-api/

REST (REPRESENTATIONAL STATE TRANSFER)

- An example
  - A proper REST response to a GET call to http://localhost:8080/employees/1 REST endpoint.


```
{
  "id": 1,
  "firstName": "Bilbo",
  "lastName": "Baggins",
  "role": "burglar",
  "name": "Bilbo Baggins",
  "_links": {
    "self": {
      "href": "http://localhost:8080/employees/1"
    },
    "employees": {
      "href": "http://localhost:8080/employees"
    }
  }
}
```


Source: https://spring.io/guides/tutorials/rest/

REST (REPRESENTATIONAL STATE TRANSFER)

- Design Principles
  - REST is designed to facilitate **strong decoupling** between the client and the server to facilitate internet-scale usage.
  - Follows a Request/ Response style of communication.
  - Some Terms:
    - Client – the consumer of services via the API
    - Server – the provider of services and the API that exposes these services
    - Resource – any content that the server responds with to a client request

REST (REPRESENTATIONAL STATE TRANSFER)

- Design Principles (contd.)
  - When a **client** requests for a **resource**, the **server** will respond with
    - the **representation** of the resource in its current **state** (Usually JSON or XML)
    - and relevant **hypermedia links (URIs)** that can be used to change the **state** of the resource

REST: SIX ARCHITECTURAL CONSTRAINTS

- Client-server architecture
- Stateless
- Cacheable
- Uniform Interface
- Layered system
- Code on demand (Optional)

Let's examine each constraint.

REST: SIX ARCHITECTURAL CONSTRAINTS

- Client-server architecture
  - Maintain the clear separation of the client and the server,
    - To promote portability of the user interface
    - To promote low coupling between the client and the server
    - To improve scalability of the server components
    - To allow clients and servers grow independently

REST: SIX ARCHITECTURAL CONSTRAINTS

- Stateless
  - Communication between clients and server must be stateless in nature.
  - Each request from client to server must contain all the information necessary to understand the request and cannot take advantage of any stored context on the server.
  - Session state is kept entirely on the client.
  - Improves visibility, reliability, and scalability.

REST: SIX ARCHITECTURAL CONSTRAINTS

- Cacheable
  - Cache constraints require that the data within a response to a request be implicitly or explicitly labeled as cacheable or non-cacheable.
  - If a response is cacheable, then a client cache is given the right to reuse that response data for later, equivalent requests.
  - Caching can eliminate requirement for some interactions thereby improving efficiency, scalability, and user-perceived performance.

REST: SIX ARCHITECTURAL CONSTRAINTS

- Uniform Interface
  - Implementations are decoupled from the services they provide, which encourages independent evolvability.
  - Continued in the next slide…

REST: SIX ARCHITECTURAL CONSTRAINTS

- Uniform Interface
  - Has four sub-constraints.
    - **Self-descriptive messages** - Each message going back and forth between the client and the server should contain enough information to process the message.
    - **Identification of resources** - Individual resources are identified in requests/ responses. This is done through URIs (Uniform Resource Identifiers). The resources themselves on the server are conceptually separate from the representations that are returned to the client.
    - **Resource manipulation through representations** - When a client holds a representation of a resource, including any metadata attached, it has enough information to modify or delete the resource's state.
    - **Hypermedia as the engine of application state** (**HATEOAS**) – continued in the next slide…

REST: SIX ARCHITECTURAL CONSTRAINTS

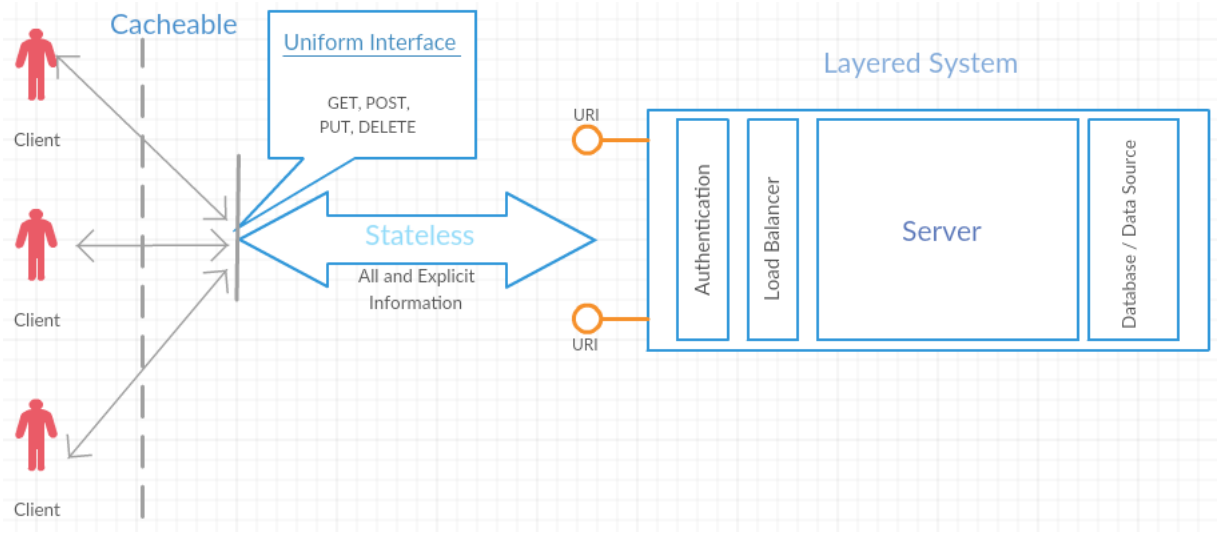
- Uniform Interface (continued…)
  - **Hypermedia as the engine of application state (HATEOAS)** - (Hey-tee-oh-s)
    - Having accessed an initial URI for the REST application a REST client should then be able to use server-provided links dynamically to discover all the available resources it needs.
    - As access proceeds, the server responds with text that includes hyperlinks to other resources that are currently available (Dynamic API).
    - There is **no need** for the client to be hard-coded with information regarding the structure of the server.
    - This is akin to a human user having to know only the main URL of a website. The rest of the URLs are discovered when consuming the site.

REST: SIX ARCHITECTURAL CONSTRAINTS

- Layered system
  - The layered system style allows an architecture to be composed of hierarchical layers by constraining component behavior such that each component cannot "see" beyond the immediate layer with which they are interacting.
  - The layered system constraints enable network-based intermediaries such as proxies and gateways to be transparently deployed between a client and server using the Web's uniform interface.
    - A network-based intermediary will intercept client-server communication for a specific purpose. Network-based intermediaries are commonly used for enforcement of security, response caching, and load balancing.

REST: SIX ARCHITECTURAL CONSTRAINTS

- Layered system (continued…)



Source: REST API Design Rulebook (Mark Masse, Oreilly, 2011)

REST: SIX ARCHITECTURAL CONSTRAINTS

- Code on demand (Optional)
  - REST allows client functionality to be extended by downloading and executing code in the form of applets or scripts.
  - This simplifies clients by reducing the number of features required to be pre-implemented.
  - This is the **only optional constraint** in REST.

TRUE REST VS. RPC (REMOTE PROCEDURE CALLS)

- Pretty URLs like /employees/10 aren't REST.
- Merely using HTTP methods such as GET, POST etc. is not REST.
- Having all the CRUD operations laid out isn't REST.

- In such an API there is no way to know **how to interact with the service** without explicit help from the developers.
  - HATEOAS constraint is missing!

TRUE REST VS. RPC (REMOTE PROCEDURE CALLS)

The response to a GET call to http://localhost:8080/employees/1 endpoint.


```
[
  {
    "id": 1,
    "name": "Bilbo Baggins",
    "role": "burglar"
  }
]
```

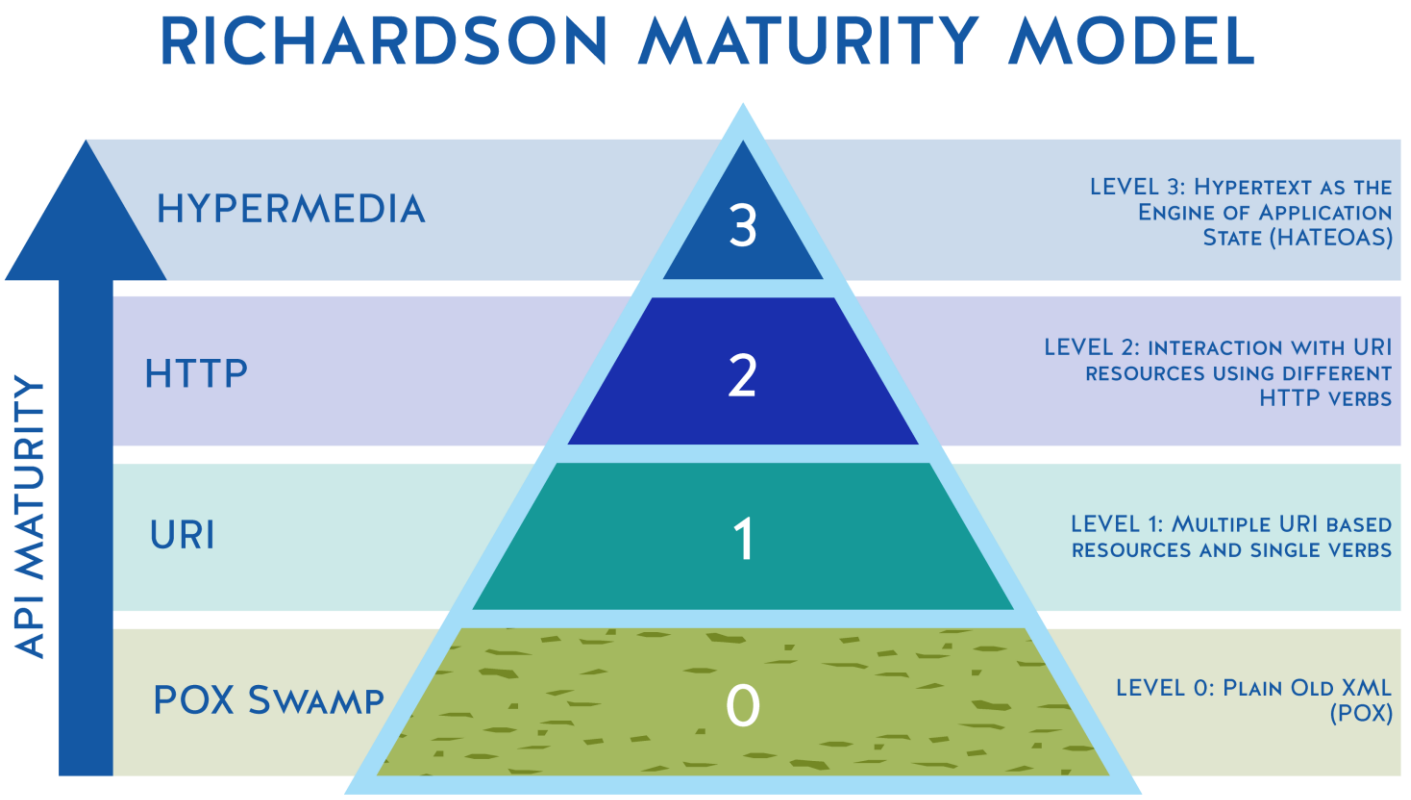

Just a JSON entity. No hyperlinks – Results in RPC


```
{
  "id": 1,
  "name": "Bilbo Baggins",
  "role": "burglar",
  "_links": {
    "self": {
      "href": "http://localhost:8080/employees/1"
    },
    "employees": {
      "href": "http://localhost:8080/employees"
    }
  }
}
```


RESTful representation of the same entity

RICHARDSON MATURITY MODEL

- It is a four-level scale that indicates extent of API conformity to the REST constraints.
- Proposed by Leonard Richardson.



Source: https://devopedia.org/richardson-maturity-model

RICHARDSON MATURITY MODEL

- Level-0: Swamp of POX
  - Usually exposes just one URI for the entire application. Uses HTTP POST for all actions, even for data fetch. SOAP or XML-RPC-based applications come under this level. POX stands for Plain Old XML. Could be JSON too.
- Level-1: Resource-Based Address/URI
  - Introduces resources and allows to make requests to individual URIs (still all typically POST) for separate actions instead of exposing one universal endpoint (API).
- Level-2: HTTP Verbs (Methods)
  - Fully utilize all HTTP commands or verbs such as GET, POST, PUT, and DELETE.
- Level-3: HATEOAS
  - Full maturity. Utilizes URI, HTTP Methods and HATEOAS.

SOME FACILITATORS OF REST

- HTTP
  - HTTP Methods – REST APIs utilize these to indicate the type of operation being performed.
    - GET – Reading a resource
    - POST – Writing a resource
    - PUT – Updating a resource
    - DELETE – Deleting a resource
  - HTTP Status Codes – REST APIs utilize these to denote the status of a response to a request.
    - 2XX – Successful responses range (e.g., 200 OK)
    - 4XX – Client errors range (e.g., 404 NOT FOUND)
    - 5XX – Server errors range (e.g., 500 INTERNAL SERVER ERROR)
    - List of HTTP status codes: HTTP response status codes

SOME FACILITATORS OF REST

- HTTP (continued…)
  - HTTP Headers – These are used to pass along additional information/ meta data between the client and the server.
    - Content-type
    - Cache-control
    - Authorization
    - List of HTTP Headers: HTTP headers
- It is important that proper HTTP methods, Headers and Status Codes are used in REST requests/ responses.

SOME FACILITATORS OF REST

- JSON (JavaScript Object Notation) – A lightweight human readable format for storing and transporting data as key-value pairs.
- HAL (Hypertext Application Language) – An open specification that provides a structure to represent RESTful resources


```
{
  "id": 1,
  "firstName": "Bilbo",
  "lastName": "Baggins",
  "role": "burglar",
  "name": "Bilbo Baggins",
  "_links": {
    "self": {
      "href": "http://localhost:8080/employees/1"
    },
    "employees": {
      "href": "http://localhost:8080/employees"
    }
  }
}
```


Source: https://spring.io/guides/tutorials/rest/

SOME BEST PRACTICES

- Use correct HTTP methods, headers and status codes according to the operation being performed.
- Accept and respond with JSON.
- Always use proper authentication and authorization mechanisms (e.g., OAuth, JWT etc.)
- Give meaningful names to the URIs.
  - E.g.,
    - https://abcd.com/users/
    - https://abcd.com/users/2
- Keep the API consistent.

MORE BEST PRACTICES

- Self study activity
  - Read and follow this list of best practices from Microsoft: RESTful web API design
  - Make sure you apply these in your assignment as well!

- More hands-on experience on REST APIs during the practical session.

SUMMARY

- Consuming web resources - Human vs. programs
- Application Programming Interface (API)
  - Examples
- REST
  - Six Architectural Constraints of REST
  - True REST vs. RPC (Remote Procedure Calls)
  - Richardson Maturity Model
- Some Facilitators of REST
    - HTTP, JSON, HAL
- Best Practices

REFERENCES

1. An Introduction to Representational State Transfer (REST)
2. R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures", University of California, Irvine, 2000
3. Mark Masse, "REST API Design Rulebook", O'Reilly Media, Inc., 2011
4. Representational state transfer
5. Richardson Maturity Model
6. RESTful web API design

THANK YOU

VISHAN.J@SLIIT.LK

NELUM.A@SLIIT.LK